ML GRIND

ML Coding Cheat Sheet

Every section of ML Grind on one sheet: the formulas, the NumPy / PyTorch idiom you will actually type, and the bug that costs people the interview. Each block links to its problems.

Notation. `N` batch / samples · `D` features or `d_model` · `T` sequence length · `h` heads, `d_k = D/h` · `V` vocab · `C` classes or channels · `K` clusters · rows are samples, so a linear layer is `X @ W` with `W: (D_in, D_out)` (PyTorch stores `(out, in)`, hence `x @ W.T`) · `log` = natural log · `⊙` elementwise · `σ` sigmoid.

PART 1 Foundations

NumPy Fundamentals

Practice ↗

Broadcasting: align shapes from the right; each pair of dims must be equal or 1.

`(N,D) + (D,) → (N,D)` · `(N,1) * (1,M) → (N,M)` · `(N,D) + (N,) →` error

Reductions: `axis=k` removes dim k. `X.sum(0)` → per column `(D,)` · `X.sum(1)` → per row `(N,)`

NUMPY

```
np.zeros((3, 4)); np.eye(3); np.full(5, 7.)
np.arange(0, 10, 2)    # [0 2 4 6 8]
np.linspace(0, 1, 5)   # 5 pts, both ends
x[1:4]; x[::-1]; x[x > 0]; x[[0, 2]]
X[:, None]             # (N,) -> (N, 1)
X.reshape(2, -1); X.T; X.swapaxes(-1, -2)
X - X.mean(0, keepdims=True) # center cols
X.argmax(axis=1)       # index per row
```

Pitfall Slices are **views**: writing to `x[1:4]` changes `x`. Boolean and fancy indexing return **copies**. `ravel()` may be a view, `flatten()` always copies. `reshape` never transposes: `(2,3).reshape(3,2)` keeps row-major order.

Python Basics for ML

Practice ↗

`A @ B: (m,k) @ (k,n) → (m,n)`, `m·k·n` multiply-adds

batched: `(…,m,k) @ (…,k,n) → (…,m,n)` · `*` is elementwise

NUMPY

```
np.sum(X)          # not builtin sum() on 2-D
A @ B              # == np.matmul(A, B)
np.dot(u, v)       # 1-D: inner product
np.outer(u, v)     # (m,), (n,) -> (m, n)
X + b              # (N,D) + (D,): per column
X + c[:, None]     # (N,D) + (N,1): per row
```

Pitfall `(N,)` vs `(N,1)`: `y_pred - y` with shapes `(N,1)` and `(N,)` silently broadcasts to `(N,N)` and the loss is wrong but finite. Assert shapes before you reduce. A Python loop over elements is ~100× slower than the vectorized call.

Einstein Summation

Practice ↗

Indices in the inputs but not after `->` are summed; the output follows the order you write.

PATTERN	MEANING	PATTERN	MEANING
`ij->`	sum	`ik,kj->ij`	matmul
`ij->j`	column sums	`i,i->`	dot
`ij->ji`	transpose	`i,j->ij`	outer
`ii->i`	diagonal	`ij,ij->`	Frobenius ⟨A,B⟩
`ii->`	trace	`bij,bjk->bik`	batched matmul

NUMPY · TORCH.EINSUM IS IDENTICAL

```
# attention scores and output, (B,h,T,dk)
S = np.einsum('bhqd,bhkd->bhqk', Q, K)
O = np.einsum('bhqk,bhkd->bhqd', A, V)
```

Pitfall Without `->` (implicit mode) the output indices are sorted alphabetically, so `einsum('ji', A)` is `A.T`. Always write the arrow. For 3+ operands pass `optimize=True`.

PyTorch Basics

Practice ↗

PYTORCH

```
model.train()
for x, y in loader:
    opt.zero_grad()
    loss = F.cross_entropy(model(x), y)
    loss.backward()              # fills p.grad
    opt.step(); sched.step()
model.eval()
with torch.no_grad():
    pred = model(x).argmax(-1)
t.detach().cpu().numpy()         # back to NumPy
```

LAYERS YOU WRITE FROM SCRATCH HERE

`GELU(x) = x·Φ(x) ≈ ½x(1 + tanh(√(2/π)(x + 0.044715x³)))`

`SiLU / Swish(x) = x·σ(x)` · `LeakyReLU = max(αx, x), α = 0.01`

`RMSNorm(x) = x / √(mean(x²) + ε) · g`

Norm stats: Batch over (N,H,W) per channel · Layer over the feature dims per sample · Instance over (H,W) per sample & channel · Group over (C/G,H,W) per sample & group

Label smoothing: `y = (1-ε)·onehot + ε/C`

LoRA: `y = xW + (α/r)(xA)B`, `A: (D_in,r)`, `B: (r,D_out) = 0` at init

Warmup + cosine: `lr = base·s/warm` for `s < warm`, else `min + ½(base-min)(1 + cos(π·prog))`

Pitfall `F.cross_entropy` wants raw **logits** `(N,C)` and int64 class indices `(N,)`: applying softmax first trains on the wrong loss. `.grad` **accumulates** until you zero it. Forgetting `model.eval()` leaves dropout and BatchNorm in training mode. `view` needs contiguous memory; `reshape` copies when it must.

PART 2 Classical ML

Data Preprocessing

Practice ↗

`Min-max: (X - min₀) / (max₀ - min₀) → [0, 1]` per column

`Z-score: (X - μ₀) / σ₀`, per column, σ with `ddof = 0`

`One-hot: np.eye(C)[y] → (N, C)`

Stratified split: per class, shuffle, put `round(frac·n_c)` in test

NUMPY

```
mu, sd = X_tr.mean(0), X_tr.std(0)
sd = np.where(sd == 0, 1.0, sd) # const col
X_tr = (X_tr - mu) / sd
X_te = (X_te - mu) / sd          # TRAIN stats
fill = np.nanmean(X_tr, axis=0)
X = np.where(np.isnan(X), fill, X)
```

Pitfall Leakage: fitting the scaler, imputer or feature selection on all data (or on the test fold) before splitting inflates every metric. Fit on train, transform the rest. `X.mean()` with no axis normalizes everything by one global number.

Supervised Learning

Practice ↗

LINEAR MODELS

`ŷ = Xw + b · MSE = (1/N)Σ(ŷ - y)²`

`∇w = (2/N)·Xᵀ(ŷ - y) · ∇b = (2/N)·Σ(ŷ - y)`

`Ridge: w = (XᵀX + λI)⁻¹Xᵀy` (don't penalize b)

Lasso step: `soft(z, λ) = sign(z)·max(|z| - λ, 0)`

`Logistic: p = σ(Xw + b) · BCE = -(1/N)Σ[y log p + (1-y) log(1-p)]`

`∇w = (1/N)·Xᵀ(p - y)` (same form as MSE, no factor 2)

`Hinge (y ∈ {-1,+1}): mean max(0, 1 - y·f(x)) + λ‖w‖²`

TREES, NEIGHBOURS, ENSEMBLES

`Gini = 1 - Σp_k² · Entropy = -Σp_k log₂ p_k`

`Gain = H(parent) - Σ (n_c/n)·H(child_c)`

`kNN: ‖a-b‖² = ‖a‖² + ‖b‖² - 2a·b`, vote of the k nearest

Gaussian NB: `argmax_c log π_c + Σ_j log N(x_j; μ_cj, σ²_cj)`

Boosting (MSE): `F₀ = ȳ`; fit `h_m` to `r = y - F_{m-1}`; `F_m = F_{m-1} + ν·h_m`

Bagging: bootstrap N rows with replacement (≈63.2% unique), average or vote

NUMPY

```
# work from logits z: stable for any z
p   = np.exp(-np.logaddexp(0, -z))  # σ(z)
bce = np.mean(np.logaddexp(0, z) - y * z)
w = np.linalg.solve(X.T @ X + lam * np.eye(D),
                    X.T @ y)
```

Pitfall `1/(1+np.exp(-z))` overflows for very negative z and `log(0) = -inf` blows up BCE: use the logit form (PyTorch: `F.binary_cross_entropy_with_logits`). Hinge needs labels ±1, BCE needs 0/1. Use `solve`, never `inv`.

PART 3 Deep Learning

Unsupervised Learning

Practice ↗

k-means: $J = \Sigma_i \|x_i - \mu_{c_i}\|^2$; assign → argmin, update → cluster mean

k-means++: next center sampled ∝ $D(x)^2$, distance to nearest chosen center

PCA: $X_c = X - \text{mean} = U\,S\,V^T$; components = $V^T[:k]$; var ratio = $S^2/\Sigma S^2$

GMM E-step: $\gamma_{ik} = \pi_k\, N(x_i|\mu_k,\Sigma_k) / \Sigma_j \pi_j\, N(x_i|\mu_j,\Sigma_j)$

M-step: $N_k = \Sigma_i \gamma_{ik}$, $\pi_k = N_k/N$, $\mu_k = \Sigma_i \gamma_{ik}\, x_i / N_k$, $\Sigma_k = \Sigma_i \gamma_{ik}\,(x_i-\mu_k)(x_i-\mu_k)^T / N_k$

DBSCAN: core if $|N_\varepsilon(x)| \geq \text{minPts}$ (x included); grow clusters from cores; border = within ε of a core; rest = noise (-1)

NUMPY

```
d2 = ((X[:, None] - C[None]) ** 2).sum(-1)
labels = d2.argmin(1)                  # (N,)
C = np.stack([X[labels == k].mean(0)
              if (labels == k).any() else C[k]
              for k in range(K)])
Xc = X - X.mean(0)
U, S, Vt = np.linalg.svd(Xc,
                         full_matrices=False)
Z = Xc @ Vt[:k].T                      # (N, k)
```

Pitfall An **empty cluster** averages nothing and returns NaN. PCA without centering finds the mean, not the variance. `np.linalg.eigh` sorts eigenvalues **ascending**, and eigenvector signs are arbitrary (compare up to ±). Run GMM in log space with `logsumexp` and add εI to Σ.

Model Evaluation

Practice ↗

Precision = TP/(TP+FP) · Recall = TPR = TP/(TP+FN)

FPR = FP/(FP+TN) · Accuracy = (TP+TN)/N

F1 = 2PR/(P+R) · $F_\beta = (1+\beta^2)PR/(\beta^2 P + R)$

ROC AUC = $P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$; 0.5 = random

AP = $\Sigma_n (R_n - R_{n-1}) \cdot P_n$ over thresholds

ECE = $\Sigma_m (|B_m|/N) \cdot |\text{acc}(B_m) - \text{conf}(B_m)|$, equal-width bins

$R^2 = 1 - SS_{res}/SS_{tot}$ · MAE = mean|y - ŷ|

NUMPY

```
# rows = true class, cols = predicted
cm = np.bincount(C * y + yhat, minlength=C * C
                 ).reshape(C, C)
pos, neg = s[y == 1], s[y == 0]
gt = (pos[:, None] > neg).mean()
auc = gt + 0.5 * (pos[:, None] == neg).mean()
```

Pitfall Accuracy hides imbalance: with 99% negatives, "always 0" scores 99%. Report PR-AUC, recall at fixed precision or F1. Guard 0/0 when nothing is predicted positive. Tune thresholds on validation, never test, and split CV folds by user/patient when rows are correlated.

Deep Learning Basics

Practice ↗

Perceptron: $\hat{y} = 1[w \cdot x + b > 0]$; on a mistake $w \mathrel{+}= \eta(y - \hat{y})x$, $b \mathrel{+}= \eta(y - \hat{y})$

Layer $Z = XW + b$. Given dZ: $dW = X^T dZ$ · $db = dZ.\text{sum}(0)$ · $dX = dZ\, W^T$

ACT.	F(X)	F′(X)	RANGE
ReLU	max(0,x)	1[x>0]	[0, ∞)
Sigmoid	1/(1+e⁻ˣ)	σ(1-σ) ≤ ¼	(0, 1)
Tanh	tanh x	1 - tanh²	(−1, 1)
GELU	x·Φ(x)	Φ + xφ	[−0.17, ∞)
SiLU	x·σ(x)	σ(1 + x(1-σ))	[−0.28, ∞)

NUMPY

```
Z1 = X @ W1 + b1; A1 = np.maximum(0, Z1)
Z2 = A1 @ W2 + b2        # dZ2 from the loss
dW2 = A1.T @ dZ2; db2 = dZ2.sum(0)
dZ1 = (dZ2 @ W2.T) * (Z1 > 0)
dW1 = X.T @ dZ1;  db1 = dZ1.sum(0)
```

Pitfall Every gradient has the **shape of what it differentiates**: if `dW.shape != W.shape`, a transpose is wrong. The bias gradient sums over the batch. A perceptron never converges on non-separable data (XOR).

Neural Networks

Practice ↗

softmax(z)ᵢ = e^(zᵢ - max z) / Σⱼ e^(zⱼ - max z)

CE = -(1/N)Σᵢ log p_{i,y_i}; ≈ ln C at init

softmax + CE (mean): dZ = (P - Y)/N

Xavier: Var(W) = 2/(fan_in + fan_out); uniform ±√(6/(fan_in + fan_out))

He (ReLU): Var(W) = 2/fan_in → std = √(2/fan_in)

BatchNorm: $\hat{x} = (x - \mu_B)/\sqrt{\sigma^2_B + \varepsilon}$, $y = \gamma\hat{x} + \beta$; running ← (1-m)·running + m·batch (m = 0.1)

Inverted dropout: $y = x \odot \text{mask}/(1-p)$, mask ~ Bernoulli(1-p); eval: y = x

NUMPY

```
def softmax(z, axis=-1):
    e = np.exp(z - z.max(axis, keepdims=True))
    return e / e.sum(axis, keepdims=True)
def log_softmax(z):
    z = z - z.max(-1, keepdims=True)
    s = np.exp(z).sum(-1, keepdims=True)
    return z - np.log(s)
loss = -log_softmax(Z)[np.arange(N), y].mean()
dZ = softmax(Z)
dZ[np.arange(N), y] -= 1; dZ /= N
```

Pitfall `np.log(softmax(z))` underflows to `-inf`; use log-softmax. Forgetting `/N` in `dZ` when the loss is a mean makes gradients N× too large. In masked CE, average over real tokens only (`ignore_index=-100`), not over all positions.

Optimization & Training

Practice ↗

SGD: θ -= lr·g · Momentum: v = μv + g; θ -= lr·v

RMSprop: s = ρs + (1-ρ)g²; θ -= lr·g/(√s + ε)

Adam: m = β₁m + (1-β₁)g; s = β₂s + (1-β₂)g²

$\hat{m} = m/(1-\beta_1{}^t)$, $\hat{s} = s/(1-\beta_2{}^t)$; θ -= lr·m̂/(√ŝ + ε) (t starts at 1)

AdamW: also θ -= lr·λ·θ, decoupled from the gradient

Global-norm clip: $n = \sqrt{\Sigma\|g_i\|^2}$; if n > c: g_i *= c/(n + 1e-6)

Loss scaling: overflow → skip step, S *= 0.5, reset counter; 2000 clean steps in a row → S *= 2

Grad accumulation: loss/k on each of k micro-batches, one step

PYTORCH

```
scaler.scale(loss).backward()
scaler.unscale_(opt)              # true grads
torch.nn.utils.clip_grad_norm_(
    model.parameters(), max_norm=1.0)
scaler.step(opt); scaler.update()
opt.zero_grad()
```

Check: grads [3, 4] and [12], c = 1 → n = 13, both scaled by 1/13. Adam's first step is ≈ lr whatever the gradient scale.

Pitfall Clipping the **scaled** gradients compares `max_norm` with S × the true norm: unscale first. Clipping each tensor separately changes the direction. An L2 term in the loss is **not** weight decay under Adam (use AdamW). Adam keeps 2 extra tensors per parameter.

CNNs & Computer Vision

Practice ↗

$H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$ (dilation d: k → d(k-1) + 1)

params = $C_{out} \cdot (C_{in}/\text{groups}) \cdot k_h \cdot k_w + C_{out}$

MACs = (weights without bias)·$H_{out} \cdot W_{out}$ per image

$out[n,o,i,j] = b_o + \Sigma_{c,a,b}$ $x[n,c,i \cdot s+a,\ j \cdot s+b] \cdot w[o,c,a,b]$ (cross-correlation)

IoU = |A∩B| / (|A| + |B| - |A∩B|) · Dice = 2|A∩B|/(|A| + |B|)

Focal: $-\alpha_t(1 - p_t)^\gamma \log p_t$ ($\alpha = 0.25$, $\gamma = 2$)

Smooth L1: $0.5x^2/\beta$ if $|x| < \beta$ else $|x| - 0.5\beta$

ResNet: y = ReLU(BN(conv(ReLU(BN(conv x)))) + shortcut(x))

NUMPY

```
# boxes are xyxy = [x1, y1, x2, y2]
def iou(a, b):
    lt = np.maximum(a[:2], b[:2])  # top-left
    rb = np.minimum(a[2:], b[2:])  # bottom-rt
    inter = np.prod(np.clip(rb - lt, 0, None))
    area = lambda z: (z[2]-z[0]) * (z[3]-z[1])
    return inter / (area(a) + area(b) - inter)
order, keep = list(np.argsort(-scores)), []
while order:                           # NMS
    i = order.pop(0); keep.append(i)
    order = [j for j in order
             if iou(B[i], B[j]) <= thr]
```

Check: 224 → conv 7×7 s2 p3 → 112 → pool 3×3 s2 p1 → 56. 3×3 conv 64→128 = 73,728 weights.

Pitfall Clamp intersection width/height at 0 or disjoint boxes "overlap". Frameworks do not flip the kernel. Mixing NCHW (PyTorch) with NHWC, or xyxy with xywh (COCO), gives plausible wrong numbers. In ResNet the final ReLU comes **after** the add.

Sequence Models

Practice ↗

RNN: h_t = tanh(x_t W_xh + h_{t-1} W_hh + b) x_t: (N,D), h: (N,H)

LSTM gates = xW_ih^T + b_ih + hW_hh^T + b_hh, split **i, f, g, o**

c′ = σ(f)⊙c + σ(i)⊙tanh(g) · h′ = σ(o)⊙tanh(c′)

GRU (PyTorch, split **r, z, n**): r, z = σ(·); n = tanh(xW_in^T + b_in + r⊙(hW_hn^T + b_hn))

h′ = (1 - z)⊙n + z⊙h (z = 1 keeps h)

Params: RNN DH + H² + H · LSTM 4(HD + H² + 2H) · GRU 3(HD + H² + 2H)

BPTT: ∂h_T/∂h_1 = Π_t diag(1 - h_t²)·W_hh^T → vanishes or explodes

NUMPY · NN.LSTMCELL LAYOUT

```
z = x @ W_ih.T + b_ih + h @ W_hh.T + b_hh
i, f, g, o = np.split(z, 4, axis=1)
c = sigmoid(f) * c + sigmoid(i) * np.tanh(g)
h = sigmoid(o) * np.tanh(c)
```

Pitfall Gate order runs without error and gives wrong numbers: PyTorch is `i,f,g,o` (LSTM) and `r,z,n` (GRU), and the GRU reset gate multiplies *after* the hidden matmul. `nn.RNN` is `(T,N,D)` unless `batch_first=True`. Clipping fixes exploding, not vanishing, gradients.

PART 4 Transformers & LLMs

Transformer Architecture

Practice ↗

Attn(Q,K,V) = softmax(QK^T/√d_k + M)·V, M_ij = -∞ for j > i (causal)

MHA: (B,T,D) → (B,T,h,d_k) → (B,h,T,d_k); attend; merge; @ W_O

Sinusoidal: PE[p,2i] = sin(p/10000^(2i/D)), PE[p,2i+1] = cos(…)

RoPE: θ_i = 10000^(-2i/d); rotate pair (x_{2i}, x_{2i+1}) by p·θ_i → q_m·k_n depends on m - n

LayerNorm: γ(x - μ)/√(σ² + ε) + β over the last dim, biased σ²

Pre-norm: x += Attn(LN(x)); x += FFN(LN(x)) (2017 paper: post-norm x = LN(x + f(x)))

GQA: h_q query heads share h_kv K/V heads; np.repeat each KV head h_q/h_kv times

Sliding window: j visible iff 0 ≤ i - j < W → O(T·W)

Backward: dV = A^T dO · dA = dO V^T · dS = A ⊙ (dA - rowsum(dA ⊙ A)) · dQ = dS K/√d_k · dK = dS^T Q/√d_k

Cost: O(T²·d) time, O(T²) memory per head · block ≈ 12D² params

NUMPY

```
def attention(Q, K, V, causal=False):
    d_k = Q.shape[-1]
    S = Q @ K.swapaxes(-1, -2) / np.sqrt(d_k)
    if causal:
        T = S.shape[-1]
        fut = np.triu(np.ones((T,T), bool), 1)
        S = np.where(fut, -np.inf, S)
    A = softmax(S, axis=-1)
    return A @ V, A
# (B,T,D) -> (B,T,h,dk) -> (B,h,T,dk)
xh = x.reshape(B, T, h, dk).transpose(0,2,1,3)
```

PYTORCH

```
F.scaled_dot_product_attention(
    q, k, v, is_causal=True)   # (B,h,T,dk)
```

Pitfall Reshaping straight to `(B,h,T,d_k)` mixes tokens across heads: reshape, **then** transpose. Scale by √d_k per head, not √D. On a 4-D NumPy array `K.T` reverses **all** axes and `K.transpose(-2,-1)` is an error (that is PyTorch syntax): use `swapaxes(-1,-2)`. A fully masked row gives NaN.

LLM Generation & Decoding

Practice ↗

Temperature: p = softmax(z/T); T < 1 sharper, ranking unchanged

Top-k: keep the k largest logits, rest -∞, softmax

Top-p: sort desc, keep the smallest prefix with cumsum ≥ p (include the crossing token)

Repetition penalty θ, each distinct generated token: z/θ if z > 0 else z·θ

Beam: score = Σ log p, keep B best; compare finished beams by score/len^α

KV cache = 2·L·h_kv·d_head·T·bytes per sequence

Speculative: accept draft x w.p. min(1, p(x)/q(x)); on reject sample norm(max(0, p - q)); all K accepted → bonus token from p

NUMPY

```
def top_p_filter(z, p):
    probs = softmax(z)
    order = np.argsort(-probs, kind='stable')
    csum = np.cumsum(probs[order])
    n = np.searchsorted(csum, p) + 1  # keep n
    out = np.full_like(z, -np.inf)
    out[order[:n]] = z[order[:n]]
    return softmax(out)
```

Check: z = [2, 1, 0.1] → T=1: [0.659, 0.242, 0.099]; T=0.5: [0.864, 0.117, 0.019]; top-p 0.8 keeps 2 tokens. LLaMA-2-7B (L=32, h_kv=32, d_head=128) at 4k tokens in fp16: **2 GiB** of KV cache.

Pitfall Temperature first, then filter logits to `-inf` and softmax (or renormalize). Dividing a **negative** logit by the penalty makes the token more likely. Beam search must add log-probs; multiplying probabilities underflows. Decoding is memory-bandwidth-bound, not FLOP-bound.

LLM Internals

Practice ↗

RMSNorm(x) = x/√(mean(x²) + ε)·w (no mean, no bias)

SwiGLU FFN = (SiLU(xW_gate) ⊙ xW_up)·W_down; d_ff ≈ (8/3)D keeps ≈ 8D² params

Online softmax per block S_b: m′ = max(m, max S_b)

l = l·e^(m-m′) + Σe^(S_b-m′) · acc = acc·e^(m-m′) + e^(S_b-m′)·V_b · out = acc/l

MoE: g = softmax(top-k router logits); y = Σ_j g_j·E_{i_j}(x)

Load balance: aux = E·Σ_e f_e·P_e (f: fraction routed, P: mean router prob)

Perplexity = exp(-(1/N)Σ_t log p(x_t | x_<t)), N = real tokens; uniform over V → V

NUMPY

```
def rmsnorm(x, w, eps=1e-6):
    ms = (x * x).mean(-1, keepdims=True)
    return x / np.sqrt(ms + eps) * w
silu = lambda z: z / (1 + np.exp(-z))
n = rmsnorm(x, w)
x = x + (silu(n @ Wg) * (n @ Wu)) @ Wd
m, l = -np.inf, 0.0          # online sm
for b in blocks:
    m2 = max(m, b.max())
    l = l*np.exp(m-m2) + np.exp(b - m2).sum()
    m = m2
```

Check: [0, 1, 5, 2, 9] in blocks of 2 → l = 1.3679, 1.0748, 1.0197. Mixtral 8×7B: ≈47B total, ≈13B active (top-2).

Pitfall When the max changes, rescale **acc as well as l**, and divide by l only at the end. Perplexity divides by real tokens (mask padding) and is only comparable under the same tokenizer. MoE: f comes from routing counts, P from the full (not top-k) softmax.

Representation Learning & Retrieval

Practice ↗

cos(a, b) = a·b/(‖a‖‖b‖); all pairs: Q̂ D̂^T with rows L2-normalized

CLIP / InfoNCE: S = Î Î^T/τ (N,N); L = ½[CE(S, diag) + CE(S^T, diag)], τ learned, init 0.07

BM25 = Σ_t IDF(t)·f(t,d)(k₁+1) / (f(t,d) + k₁(1 - b + b|d|/avgdl))

IDF(t) = ln((N - n_t + 0.5)/(n_t + 0.5) + 1); k₁ ≈ 1.2–2, b ≈ 0.75

Recall@k = relevant in top k / all relevant · MRR = mean 1/rank of first hit

Hybrid: min-max normalize, α·bm25 + (1-α)·cos; or RRF Σ 1/(60 + rank)

NUMPY

```
def l2n(x):                        # rows
    return x / np.linalg.norm(x, axis=1,
                              keepdims=True)
S = l2n(img) @ l2n(txt).T / tau    # (N, N)
i = np.arange(N)
ce = lambda L: -log_softmax(L)[i, i].mean()
loss = 0.5 * (ce(S) + ce(S.T))
top = np.argsort(-scores, kind='stable')[:k]
```

Check: two perfectly aligned pairs at τ = 1 still give loss 0.3133; smaller τ makes the same gap matter more.

Pitfall Normalize **rows** (axis=1), not columns. `argsort(s)[::-1]` reverses ties; sort `-s` stably. Raw BM25 is unbounded and cosine is in [-1, 1]: normalize before fusing. Small τ overflows without max-subtraction.

PART 5 Generative Models & RL

Generative Models

Practice ↗

KL(P‖Q) = Σ P log(P/Q) ≥ 0, = 0 iff P = Q, not symmetric

KL(N(μ_p,σ_p²)‖N(μ_q,σ_q²)) = log(σ_q/σ_p) + (σ_p² + (μ_p-μ_q)²)/(2σ_q²) - ½

VAE: z = μ + exp(½·logσ²)⊙ε, ε ~ N(0, I)

KL_i = -½Σ_j(1 + logσ² - μ² - σ²); loss = recon + mean over batch of KL_i

VQ: e = nearest code; L = ‖sg[z_e] - e‖² + β‖z_e - sg[e]‖²; z_q = z_e + sg(e - z_e)

DDPM: α_t = 1 - β_t, ᾱ_t = Π_{s≤t} α_s; x_t = √ᾱ_t·x₀ + √(1-ᾱ_t)·ε; L = ‖ε - ε_θ(x_t,t)‖²

Reverse: x_{t-1} = (x_t - β_t/√(1-ᾱ_t)·ε_θ)/√α_t + √β_t·z (no z at t = 0)

CFG: ε̂ = ε_u + w(ε_c - ε_u); w = 1 is unguided

Flow matching: x_t = (1-t)x₀ + t·x₁ (x₀ noise, x₁ data), target v = x₁ - x₀; sample x += Δt·v_θ(x,t)

NUMPY

```
betas = np.linspace(1e-4, 0.02, T)
abar = np.cumprod(1 - betas)   # t from 0
a = abar[t].reshape(-1, *[1] * (x0.ndim - 1))
eps = np.random.randn(*x0.shape)
xt = np.sqrt(a) * x0 + np.sqrt(1 - a) * eps
kl = (-0.5 * (1 + lv - mu**2 - np.exp(lv))
      ).sum(axis=1).mean()
```

Check: μ = 1, σ = 0.5 → KL = 0.8181. β = [0.1, 0.2, 0.3] → ᾱ = [0.9, 0.72, 0.504].

Pitfall `exp(log_var)` is the **variance**; the sample needs `exp(0.5·log_var)`. Averaging KL over (batch, latent) divides it by latent_dim. The forward process uses √ᾱ, not ᾱ. Decide 0- vs 1-indexed t once and index β and ᾱ the same way.

Reinforcement Learning

Practice ↗

```
G_t = r_t + γG_{t+1} · V(s) = Σ_a π(a|s) Σ_s' P(s'|s,a)[r + γV(s')]
Q-learning: Q += α[r + γ·max_a' Q(s',a')·(1-done) - Q] (off-policy)
SARSA: target r + γQ(s',a') with the a' actually taken (on-policy)
TD(0): V(s) += α[r + γV(s')(1-done) - V(s)]
Value iteration: V(s) ← max_a Σ_s' P(s'|s,a)[r + γV(s')]
n-step: Σ_{k<n} γ^k r_{t+k} + γ^n V(s_{t+n})
REINFORCE: ∇J = E[Σ_t G_t ∇log π(a_t|s_t)]; softmax: ∇_z log π(a) = onehot(a) - π
GAE: δ_t = r_t + γV_{t+1}(1-d_t) - V_t; A_t = δ_t + γλ(1-d_t)A_{t+1}
PPO: ρ = e^(logπ - logπ_old); J = E[min(ρA, clip(ρ, 1-ε, 1+ε)A)], ε = 0.2
Reward model: L = -log σ(r_chosen - r_rejected)
DPO: L = -log σ(β[(logπ_c - logπ_ref,c) - (logπ_r - logπ_ref,r)])
RLHF reward: r - β·Σ_t kl_t, kl_t = logπ_θ - logπ_ref (k1)
```

NUMPY

```
def gae(r, v, done, g=0.99, lam=0.95):
    # len(v) = len(r) + 1 (bootstrap value)
    A, last = np.zeros(len(r)), 0.0
    for t in reversed(range(len(r))):
        nd = 1.0 - done[t]
        d = r[t] + g * v[t + 1] * nd - v[t]
        A[t] = last = d + g * lam * nd * last
    return A        # returns = A + v[:-1]
logsig = lambda x: -np.logaddexp(0, -x)
```

Check: Q = 1, r = 5, Q(s') = [3, 4], γ = 0.9, α = 0.5 → 4.8 (SARSA with a' = 0 → 4.35).

Pitfall Don't bootstrap through a **terminal** state, but do through a time-limit truncation. Objectives are maximized: loss = −J. `log(sigmoid(x))` underflows; use `logaddexp` or `F.logsigmoid`. Put the KL in the reward **or** in the loss, never both.

Post-training & Efficiency

Practice ↗

```
GRPO: A_i = (r_i - mean r)/(std r + ε) over the G samples of one prompt (population std)
k3 KL per token: e^(-kl_t) + kl_t - 1 ≥ 0, unbiased
J = mean_i (1/|o_i|)Σ_t [min(ρ_t A_i, clip(ρ_t)A_i) - β·k3_t]
Int8 absmax, one scale per output row: s = max|w|/127; q = clip(round(w/s), -127, 127); ŵ = q·s; |w - ŵ| ≤ s/2
LoRA trainable params: r(D_in + D_out) vs D_in·D_out
```

METHOD	NEEDS	DATA
PPO-RLHF	reward model + critic	online samples
DPO	reference model only	offline pairs
GRPO	reward/verifier, no critic	G samples per prompt

NUMPY

```
R = rewards.reshape(-1, G)   # (prompts,G)
A = (R - R.mean(1, keepdims=True)) / (
     R.std(1, keepdims=True) + 1e-8)
# W: (out, in)
amax = np.abs(W).max(1, keepdims=True)
s = np.maximum(amax, 1e-8) / 127
q = np.clip(np.round(W / s), -127, 127)
q = q.astype(np.int8)
```

Check: rewards [1, 0, 0, 1] → A = [1, −1, −1, 1]; [1, 0, 0, 0] → 1.732 and −0.577. Row [0.7, −1.2, 0.3] → q = [74, −127, 32].

Pitfall Use `ddof=0` and add ε: a group with identical rewards must give A = 0, not NaN (and it contributes no gradient). Average token terms over real tokens only. Accumulate int8 products in **int32**. One per-tensor scale lets a single outlier erase small rows.

PART 6 Capstone

End-to-End Implementations

Practice ↗

NUMPY · MINI GPT FORWARD

```
x = tok_emb[ids] + pos_emb[:T]      # (B,T,D)
for blk in blocks:                   # pre-norm
    x = x + blk.attn(blk.ln1(x), causal=True)
    x = x + blk.mlp(blk.ln2(x))
logits = ln_f(x) @ W_vocab           # (B,T,V)
loss = ce(logits[:, :-1], ids[:, 1:]) # shift
```

```
ViT patches: (B,C,H,W) → (B, HW/P², C·P²) → linear → prepend CLS → + pos
Grad check: (f(θ+h) - f(θ-h))/2h, h ≈ 1e-5 in float64; rel. error |a-n|/max(|a|,|n|) < 1e-6
```

SANITY CHECKS BEFORE YOU TRUST A MODEL

- Loss at init ≈ ln C (10 classes: 2.303; GPT-2 vocab: 10.82).
- It can overfit one small batch to ≈ 0 loss.
- Shapes printed at every layer of the first forward pass.

BUG-HUNT CHECKLIST

- Causal mask missing (loss suspiciously low) or masking the diagonal.
- Scale √D instead of √d_k; softmax over the wrong axis.
- Targets not shifted by one, so the model learns to copy.
- Residual dropped, dropout left on in eval, LayerNorm over the batch axis.

Pitfall The capstones reuse every earlier block: when one fails, unit-test the piece (attention, LayerNorm, the loss) against its section before debugging the whole model.

Numbers to Know

mlgrind.dev ↗

ln 2 · ln 10 · e	0.693 · 2.303 · 2.718
σ(0) · σ′(0) · σ(2)	0.5 · 0.25 · 0.881
Transformer params	≈ 12·L·D² + V·D
GPT-2 small (L=12, D=768)	85M blocks + 39M embeddings ≈ 124M
Training FLOPs	≈ 6 · params · tokens
Inference FLOPs / token	≈ 2 · params
Chinchilla-optimal data	≈ 20 tokens per parameter
Bytes / param	fp32 4 · bf16 2 · int8 1 · int4 0.5
Mixed-precision Adam	≈ 16 B/param (2 + 2 + 4 master + 4 + 4)
7B weights in fp16	≈ 14 GB
Bootstrap unique rows	1 − 1/e ≈ 63.2%
LoRA r=8 on 4096×4096	65,536 vs 16.8M params
Attention / MLP cost	O(T²·D) vs O(T·D²) per layer